



به نام خدا
دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر



مهسا ابرغانی اقدام - ۸۱۰۱۰۳۰۵۳ - تمرین سوم - درس سامانه‌های یادگیری ماشین توزیع شده

سوال (۱)

الف) توضیح کد:

```
# الف-1
import sys
from operator import add
from pyspark.sql import SparkSession
import time
startTime = time.time()

if __name__ == "__main__":

    spark = SparkSession\
        .builder\
        .appName("PythonWordCountWithParallelize")\
        .getOrCreate()

    path = 'shahname.txt'
    with open(path, 'r') as file:
        lines = file.readlines()

    rdd = spark.sparkContext.parallelize(lines)
    final_lines = rdd.filter(lambda line: not line.strip().startswith('#') and line.strip() != "")

    line_counts = final_lines.map(lambda line: 1).reduce(add)
    print(f'Total lines: {line_counts}')

    words = final_lines.flatMap(lambda line: line.split())
    word_counts = words.map(lambda word: 1).reduce(add)
    print(f'Total words: {word_counts}')

    distinct_words = words.distinct()
    num_distinct_words = distinct_words.map(lambda word: 1).reduce(add)
    print(f'Total distinct words: {num_distinct_words}')

    spark.stop()

endTime = time.time()
```

۱. خواندن داده‌ها: فایل متنی شاهنامه خوانده و به RDD تبدیل شد.
۲. فیلتر کردن خطوط نامعتبر: خطوط خالی و خطوطی که با # شروع می‌شوند، حذف شدند.
۳. محاسبه تعداد کل ابیات: هر خط معتبر به عنوان یک بیت در نظر گرفته شده و تعداد کل خطوط محاسبه شد.

۴. محاسبه تعداد کل کلمات: هر خط به کلمات جداگانه شکسته و تعداد کلمات محاسبه شد.

۵. محاسبه تعداد کلمات یکتا: کلمات بدون تکرار استخراج و تعداد آنها شمارش شد.

۶. زمان اجرا: زمان کلی اجرای برنامه نیز اندازه‌گیری شد.

نتایج:

تعداد کل ابیات: 22845	•	 Total lines: 22845
تعداد کل کلمات: 261190	•	Total words: 261190
تعداد کل کلمات یکتا: 12535	•	Total distinct words: 12535
زمان اجرا: 6.93 ثانیه	•	Total time: 6.931774139404297 seconds

(ب) یافتن ۱۰ قافیه پرتکرار در متن شاهنامه و نمایش تعداد تکرار هر قافیه:

```
#1 - ب
import sys
from operator import add
from pyspark.sql import SparkSession
import time
startTime = time.time()

spark = SparkSession\
    .builder \
    .appName("PythonWordCount") \
    .getOrCreate()

path = 'shahname.txt'

lines = spark.read.text(path).rdd.map(lambda r: r[0])
#ابیات
final_lines = lines.filter(lambda line: not line.strip().startswith('#') and line.strip() != "")
split_lines = final_lines.flatMap(lambda line: line.split("\t"))

outputMesra = 'output.txt'
with open(outputMesra, 'w') as f:
    for i in split_lines.collect():
        f.write(i + '\n')

outputGafie = 'output_gafie.txt'
gafieh = spark.read.text(outputMesra).rdd.map(lambda line: line[0].split()[-1] if line[0] else '')
with open(outputGafie, 'w') as g:
    for i in gafieh.collect():
        g.write(i + '\n')
```

```
GafiehHa = spark.read.text('output_gafie.txt')
counts = GafiehHa.rdd.flatMap(lambda x: x[0].split(' ')) \
    .map(lambda x: (x, 1)) \
    .reduceByKey(add)
NewOutput = counts.collect()

repeats = []
for (word, count) in NewOutput:
    repeats.append({'count': count, 'word': word})

sortedRepeatsOrderCount = sorted(repeats, key = lambda item : item['count'], reverse=True)

for i in range(10):
    print(f'top {i+1} is : {sortedRepeatsOrderCount[i]}')
endTime = time.time()
totalTime = endTime - startTime
print(f"Total time: {totalTime} seconds")
```

توضیح عملکرد کد:

۱. خطوط ورودی: فایل شاهنامه خوانده شده و خطوطی که خالی هستند یا با # شروع می‌شوند، حذف شدند.
۲. جداسازی مصراع‌ها: خطوط به مصراع‌های جداگانه تقسیم شده و در فایل output.txt ذخیره شدند.
۳. استخراج قافیه‌ها: آخرین کلمه هر مصراع به عنوان قافیه استخراج و در فایل output_gafie.txt ذخیره شد.
۴. محاسبه تعداد تکرار قافیه‌ها: تعداد تکرار هر قافیه محاسبه و نتیجه به ترتیب نزولی مرتب شد.
۵. نمایش ۱۰ قافیه پرتکرار: پرتکرارترین قافیه‌ها همراه با تعداد نمایش داده شدند.
۶. زمان اجرا: مدت زمان کل اجرای برنامه محاسبه شد.

نتایج: ۱۰ قافیه پرتکرار و تعداد تکرار آن‌ها:

```
→ top 1 is : {'count': 1131, 'word': 'بود'}
top 2 is : {'count': 1130, 'word': 'شاه'}
top 3 is : {'count': 1046, 'word': 'سیاه'}
top 4 is : {'count': 700, 'word': 'اوی'}
top 5 is : {'count': 584, 'word': 'شیریار'}
top 6 is : {'count': 559, 'word': 'گرد'}
top 7 is : {'count': 556, 'word': 'راه'}
top 8 is : {'count': 536, 'word': 'را'}
top 9 is : {'count': 511, 'word': 'خند'}
top 10 is : {'count': 466, 'word': 'من'}
Total time: 10.53723692893982 seconds
```

(ج) در این بخش، از تکنیک N-Gram برای استخراج سه‌گانه‌های متوالی کلمات از متن شاهنامه استفاده شده است. در ابتدا، با استفاده از یک پردازش توزیع‌شده در Apache Spark، سه‌گانه‌های کلمات استخراج شده و سپس تعداد تکرار هر سه‌گانه محاسبه گردید. نتایج به صورت ۱۰ سه‌گانه پرتکرار نمایش داده شده است.

توضیح کد:

```

# 1- ج
from pyspark.sql import SparkSession
from pyspark.ml.feature import NGram
from pyspark.sql.functions import col, split, monotonically_increasing_id, explode
import time
startTime = time.time()

if __name__ == "__main__":
    spark = SparkSession.builder.appName("NGramExample").getOrCreate()

    df = spark.read.text("output.txt")

    wordDataFrame = df.withColumn("words", split(col("value"), " ")) \
        .withColumn("id", monotonically_increasing_id()) \
        .select("id", "words")

    ngram = NGram(n=3, inputCol="words", outputCol="ngrams")
    ngramDataFrame = ngram.transform(wordDataFrame)

    ngramFlattened = ngramDataFrame.withColumn("ngram", explode("ngrams"))
    ngramCount = ngramFlattened.groupBy("ngram").count().orderBy("count", ascending=False)

    ngramList = ngramCount.collect()

    for i in range(10):
        print(f"top {i + 1} : ' {ngramList[i]['ngram']}' with count {ngramList[i]['count']}")

    spark.stop()
    endTime = time.time()
    totalTime = endTime - startTime
    print(f"Total time: {totalTime} seconds")

```

۱. بارگذاری داده‌ها: داده‌های متن شاهنامه از فایل output.txt بارگذاری شد.
۲. تقسیم‌بندی متن به کلمات: برای تقسیم متن به کلمات، از تابع split در PySpark استفاده شد که کلمات را از متن جدا می‌کند.
۳. ایجاد NGram-3: از کلاس NGram در PySpark استفاده شد تا سه‌گانه‌های متوالی از کلمات استخراج گردد. این عملیات بر روی ستون words که شامل کلمات متن است انجام شد و نتایج در ستون جدید ngrams ذخیره گردید.
۴. تعدادگیری و مرتب‌سازی: سه‌گانه‌های به‌دست‌آمده با استفاده از تابع groupBy و count شمارش شدند و سپس بر اساس تعداد تکرار مرتب شدند.
۵. نمایش نتایج: ده سه‌گانه پر تکرار با تعداد تکرار آن‌ها نمایش داده شدند.
۶. محاسبه زمان اجرا

نتایج:

در زیر، ۱۰ سه‌گانه پر تکرار که از متن شاهنامه استخراج شدند و تعداد تکرار آن‌ها آورده شده است:

```

top 1 : 'چنین داد پاسخ' with count 306
top 2 : 'داد پاسخ که' with count 231
top 3 : 'چنین گفت کای' with count 134
top 4 : 'هر او را' with count 126
top 5 : 'چنین گفت با' with count 124
top 6 : 'تاج و تخت' with count 109
top 7 : 'بدو گفت کای' with count 108
top 8 : 'ر هر سو' with count 92
top 9 : 'نخست از بر' with count 85
top 10 : 'تو گیتی که' with count 70
Total time: 18.562411069869995 seconds

```

سوال ۲)

الف) الگوریتم word2vec در حالت Skip-gram

در این بخش، الگوریتم Skip-gram برای مدل‌سازی معنایی کلمات در متن شاهنامه پیاده‌سازی شد. مراحل کار براساس کد نوشته شده به صورت زیر بود:

۱. پیش‌پردازش متن:

- فایل متنی شاهنامه خوانده و نویسه‌های غیرمجاز و کلمات توقف فارسی از آن حذف شد.
- خطوط تمیز شده به RDD تبدیل شدند و از آنها برای استخراج جفت‌های کلمه مرکزی و زمینه استفاده شد.

۲. ایجاد جفت‌های Skip-gram :

- با استفاده از اندازه پنجره ۲، جفت‌هایی از کلمات مرکزی و زمینه تولید شد.

۳. آموزش مدل:

- دو ماتریس وزن w و w_1 به صورت تصادفی مقداردهی اولیه شدند.
- فرآیند آموزش شامل مراحل feed-forward و backpropagation بود.
- مدل در ۱۰ دوره (epoch) اجرا شد، و بردارهای معنایی (embedding words) برای کلمات به دست آمدند.

نتایج تست: کلمات مشابه با «رستم» که مدل Skip-gram پیش‌بینی کرد، عبارت بودند از:

```

2 alef test :['اندراور', 'باخوش', 'رجمشید', 'رستم', 'بجا']

```

ب) استخراج embedding کلمات مشخص شده:

تابع `get_word_embedding`:

- این تابع برای گرفتن بردار `embedding` کلمه‌ای مشخص استفاده می‌شود.
- اگر کلمه در دیکشنری `word_to_idx` (نگاشت کلمه به ایندکس) وجود نداشته باشد، پیام خطا نمایش داده می‌شود و مقدار `None` برمی‌گردد.
- اگر کلمه موجود باشد، ایندکس آن از دیکشنری استخراج می‌شود و بردار مربوطه از ماتریس وزن W بازگردانده می‌شود.

ایجاد دیکشنری `embeddings`:

```
embeddings = {word: get_word_embedding(word) for word in words_to_extract}
```

برای هر کلمه در لیست `words_to_extract`، تابع `get_word_embedding` فراخوانی می‌شود.

نتیجه به صورت دیکشنری ذخیره می‌شود که کلید آن کلمه و مقدار آن بردار `embedding` کلمه است.

تعریف `schema` برای `DataFrame`:

اسکیمای یک `DataFrame` مشخص می‌شود:

`Word`: نام کلمه به صورت رشته‌ای.

`embedding`: بردار `embedding` کلمه، که یک آرایه از اعداد اعشاری (`Float`) است.

ایجاد `RDD` برای `DataFrame`

- بردارهای `embedding` به همراه کلمات به قالب `RDD` تبدیل می‌شوند.
- از `Row` برای تعریف هر ردیف استفاده شده است.
- کلماتی که بردار `embedding` آنها `None` است، حذف می‌شوند.
- برای استفاده از بردارها در `DataFrame`، آنها به فرمت لیست تبدیل می‌شوند.

ایجاد `DataFrame`

یک `DataFrame` از `RDD` تولید می‌شود که شامل دو ستون است:

- نام کلمه
- بردار تعبیه کلمه

```
#2-ب
from pyspark.sql import Row

def get_word_embedding(word):
    if word not in word_to_idx:
        print(f"{word} not found in vocabulary!")
        return None
    idx = word_to_idx[word]
    return W[idx]

words_to_extract = ["زال", "رخش", "اسفندیار", "سهراب", "رستم"]

embeddings = {word: get_word_embedding(word) for word in words_to_extract}

schema = StructType([
    StructField("word", StringType(), True),
    StructField("embedding", ArrayType(FloatType(), True)
])

embeddings_rdd = spark.sparkContext.parallelize([
    Row(word=word, embedding=embedding.tolist())
    for word, embedding in embeddings.items() if embedding is not None
])

embeddings_df = spark.createDataFrame(embeddings_rdd)

embeddings_df.show(truncate=False)
```

بردارهای معنایی کلمات رستم، سهراب، اسفندیار، رخش، زال استخراج شد. این بردارها در زیر نمایش داده می‌شوند:

```
word      embedding
-----
[0.2115700544655983, 0.05336777210945021, 0.5864609100964526, 0.5943430543611025, 0.7543252439107251, 0.06455700225210894, 0.5076334344289668, 0.12812514294001345, 0.458907034061292, 0.7447191472175341,
[0.662166720618675, 0.1629042414252417, 0.5696269940154723, 0.6116594129044539, 0.0632385153850638, 0.16189946439997682, 0.16733386288513208, 0.2047495861355944, 0.11989218904258703, 0.4381875511674162,
[0.09588239674933363, 0.7955871317613106, 0.6381332920960446, 0.7380424274285174, 0.6915044357179897, 0.49508753219168933, 0.0015452185781983374, 0.710526265917706, 0.0855550730004383, 0.7428009640616,
[0.2290094318908542, 0.11591889788148446, 0.13084108247605108, 0.51237080898082841, 0.11606594604503628, 0.633084693272411, 0.5467299270539021, 0.5723660866662992, 0.6281834016230237, 0.4567151603555262],
[0.5044174436368487, 0.2805445637850904, 0.2311543670007144, 0.30637213480173164, 0.5894755404964354, 0.31275350709267946, 0.36677498546775134, 0.7580499364485729, 0.47515604136527795, 0.319282758868970]
```

ج) محاسبه میزان شباهت و نمایش نمودار Heatmap

۱. شباهت کسینوسی بین بردارهای تعبیه کلمات محاسبه شد.
۲. مقادیر شباهت به صورت یک ماتریس ارائه شدند و در قالب یک نمودار Heatmap بصری‌سازی شدند.

```

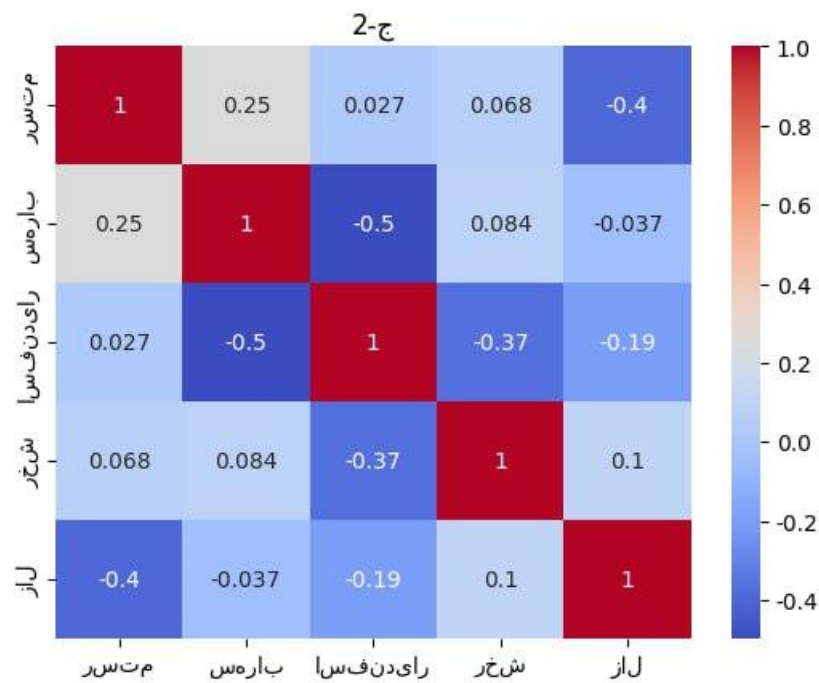
#ج-2
def cosine_similarity(vec1, vec2):
    return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(vec2))

similarity_matrix = []
words_list = list(embeddings.keys())
for word1 in words_list:
    row = []
    for word2 in words_list:
        similarity = cosine_similarity(embeddings[word1], embeddings[word2])
        row.append(similarity)
    similarity_matrix.append(row)

sns.heatmap(similarity_matrix, xticklabels=words_list, yticklabels=words_list, annot=True, cmap="coolwarm")
plt.title('ج-2')
plt.show()

```

نتایج: این نمودار روابط معنایی بین کلمات را به صورت نسبی نمایش می دهد.



د) کاهش ابعاد با PCA

```

#2-۳
from pyspark.ml.linalg import Vectors
from pyspark.ml.feature import PCA

pca_data = [
    (Vectors.dense(embedding.toList()), word)
    for word, embedding in embeddings.items() if embedding is not None
]

pca_df = spark.createDataFrame(pca_data, ["features", "word"])

pca = PCA(k=2, inputCol="features", outputCol="pcaFeatures")
model = pca.fit(pca_df)
result = model.transform(pca_df).select("word", "pcaFeatures")

pca_result = result.collect()
words = [row["word"] for row in pca_result]
pca_features = np.array([row["pcaFeatures"] for row in pca_result])

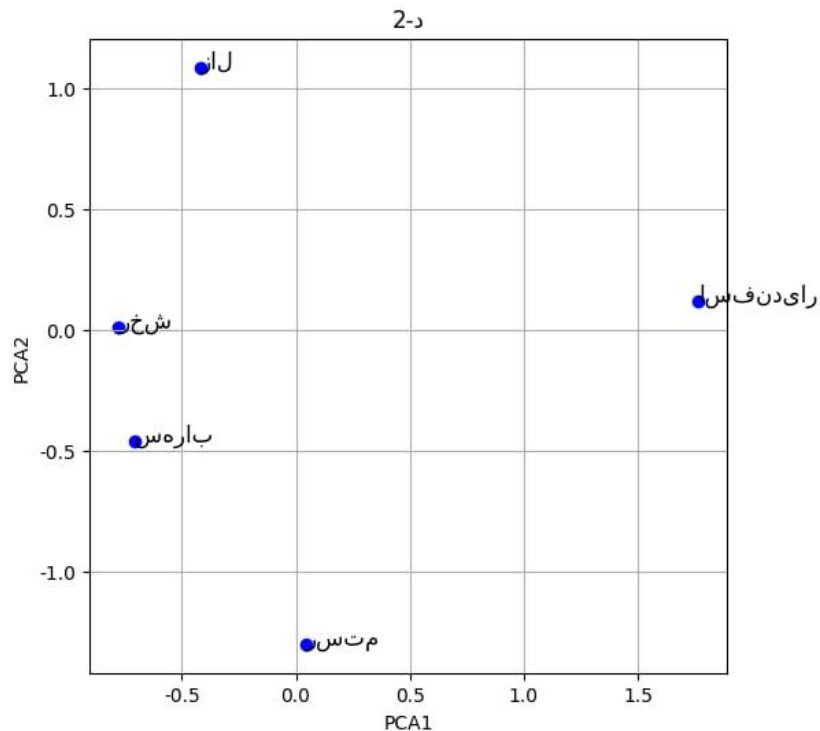
plt.figure(figsize=(6, 6))
plt.scatter(pca_features[:, 0], pca_features[:, 1], color="blue")

for i, word in enumerate(words):
    plt.text(pca_features[i, 0], pca_features[i, 1], word, fontsize=12)
plt.title('2-۳')
plt.xlabel("PCA1")
plt.ylabel("PCA2")
plt.grid()
plt.show()

spark.stop()

```

۱. بردارهای معنایی استخراج شده با استفاده از الگوریتم PCA به دو بُعد کاهش یافتند.
۲. نتایج در یک نمودار دوبعدی نمایش داده شد، که هر نقطه نشان‌دهنده مکان کلمه‌ای خاص در فضای معنایی دو بعدی بود.



زمان اجرا

کل زمان اجرای کد: ۱۶.۹۳ ثانیه

سوال (۳)

الف) در این بخش، یک پوشه جدید با استفاده از شماره دانشجویی در HDFS ایجاد شده و فایل مربوطه در آن آپلود گردید.

کد اجرا شده (به ترتیب ایجاد پوشه و انتقال (آپلود) فایل (ها):

```
# HDFS ایجاد پوشه با شماره دانشجویی در
```

```
hdfs dfs -mkdir /810103053
```

```
# انتقال فایل موردنظر به پوشه ایجادشده
```

```
hdfs dfs -put /home/dmls/aqdam/k-means.py /810103053
```

```
hdfs dfs -put /home/dmls/aqdam/customers.csv /810103053
```

توضیح عملکرد کد:

۱. دستور `hdfs dfs -mkdir /810103053` پوشه‌ای با نام شماره دانشجویی در HDFS ایجاد می‌کند.

۲. دستور `hdfs dfs -put` فایل `k-means.py` و `customers.csv` را از مسیر محلی

`/home/dmls/aqdam/` به پوشه‌ای که در HDFS ایجاد شده، منتقل می‌کند.

تصویر نتایج کسب شده از کلاستر:

```
aqdam@raspberrypi-dml0:~$ hdfs dfs -ls /
Found 34 items
drwxr-xr-x - salemi supergroup 0 2023-12-31 11:04 /810101182
drwxr-xr-x - attar supergroup 0 2024-01-01 12:02 /810101229
drwxr-xr-x - kamahi supergroup 0 2023-12-25 18:56 /810101254
drwxr-xr-x - moradi supergroup 0 2024-01-02 17:18 /810101266
drwxr-xr-x - akbari supergroup 0 2024-12-26 19:15 /810102004
drwxr-xr-x - jamildehi supergroup 0 2024-12-20 12:27 /810102106
drwxr-xr-x - hosseini supergroup 0 2024-12-27 16:55 /810102121
drwxr-xr-x - khorramfar supergroup 0 2024-12-27 06:41 /810102129
drwxr-xr-x - rahimi supergroup 0 2024-01-02 22:27 /810102151
drwxr-xr-x - ramezani supergroup 0 2024-12-24 17:03 /810102158
drwxr-xr-x - sardari supergroup 0 2024-12-25 18:16 /810102170
drwxr-xr-x - vadiati supergroup 0 2024-12-26 12:52 /810102287
drwxr-xr-x - aqdam supergroup 0 2024-12-27 15:20 /810103053
drwxr-xr-x - molaee supergroup 0 2024-12-26 15:52 /810103241
```

```
aqdam@raspberrypi-dml0:~$ hdfs dfs -ls /810103053
Found 3 items
drwxr-xr-x - aqdam supergroup 0 2024-12-27 14:15 /810103053/aqdam
-rw-r--r-- 1 aqdam supergroup 3981 2024-12-26 16:59 /810103053/customers.csv
-rw-r--r-- 1 aqdam supergroup 2796 2024-12-27 15:20 /810103053/k-means.py
aqdam@raspberrypi-dml0:~$ |
```

ب) محاسبه اطلاعات آماری از جمله حداقل، حداکثر، میانگین و واریانس درآمد سالانه مشتریان با استفاده از PySpark : کد نوشته شده در زیر را با دستور `spark-submit k-means.py` اجرا کردیم. ابتدا کد نوشته شده را توضیح داده و سپس نتایج را گزارش خواهیم کرد.

```

from pyspark.sql.functions import min, max, avg, variance, round
from pyspark.sql import SparkSession
import time

if __name__ == "__main__":
    startTime = time.time()
    spark = SparkSession\
        .builder\
        .appName("soal-3-aqdam")\
        .getOrCreate()

    # مسیر فایل داده ها
    path = "customers.csv"

    # خواندن فایل CSV
    data = spark.read.csv(path, header=True, inferSchema=True)

    # ستون موردنظر برای تحلیل
    income = "Annual Income (k$)"

    # انتخاب ستون درآمد سالانه
    income_df = data.select("Annual Income (k$)")

    # محاسبه اطلاعات آماری
    result = income_df.select(
        min(income).alias("Min_Income"),
        max(income).alias("Max_Income"),
        avg(income).alias("Avr_Income"),
        round(variance(income), 3).alias("Var_Income")
    )

    # نمایش نتایج
    result.show()

```

توضیح کد:

۱. ایجاد سشن **PySpark** : با استفاده از `SparkSession`، یک سشن برای اجرای کد `PySpark` ایجاد شد.
۲. خواندن داده‌ها: فایل `customers.csv` که اطلاعات مشتریان را شامل می‌شود، در قالب `DataFrame` خوانده شد.
۳. انتخاب ستون درآمد سالانه: ستون مربوط به درآمد سالانه (`Annual Income (k$)`) جدا شد تا روی آن تحلیل انجام شود.

۴. محاسبه مقادیر آماری: مقادیر حداقل (min)، حداکثر (max)، میانگین (avg) و واریانس (variance) درآمد محاسبه شد.

۵. نمایش نتایج: نتایج نهایی به صورت جدول در خروجی نمایش داده شدند.

Min_Income	Max_Income	Avr_Income	Var_Income
15	137	60.56	689.836

تصویر نتایج کسب شده از کلاستر:

```
24/12/27 15:28:43 INFO TaskSchedulerImpl: Kitting all running tasks in stage 4. Stage finished
24/12/27 15:28:43 INFO DAGScheduler: Job 3 finished: showString at NativeMethodAccessorImpl.java:0, took 0.891752 s
24/12/27 15:28:44 INFO CodeGenerator: Code generated in 78.026617 ms
+-----+-----+-----+-----+
|Min_Income|Max_Income|Avr_Income|Var_Income|
+-----+-----+-----+-----+
|      15|      137|     60.56|    689.836|
+-----+-----+-----+-----+
```

ج) توضیح کد:

۱. انتخاب ستون‌ها: ستون‌های Annual Income (k\$) و Spending Score (1-100) به عنوان ویژگی‌های ورودی برای خوشه‌بندی انتخاب شدند.
۲. تبدیل داده‌ها: با استفاده از VectorAssembler، داده‌ها به بردار ویژگی‌ها تبدیل شدند.
۳. استانداردسازی داده‌ها: داده‌ها با استفاده از StandardScaler استاندارد شدند تا اثر مقیاس متغیرها کاهش یابد.
۴. خوشه‌بندی K-Means: مدل K-Means با تعداد ۳ خوشه (k=3) بر روی داده‌ها اعمال شد.
۵. ارزیابی مدل: امتیاز سیلوئت برای ارزیابی کیفیت خوشه‌بندی محاسبه شد.
۶. مراکز خوشه‌ها: مراکز خوشه‌های تشکیل‌شده محاسبه و نمایش داده شدند.
۷. پیش‌بینی خوشه‌ها: داده‌ها با برچسب خوشه‌های متناظر در خروجی نمایش داده شدند.

```

from pyspark.sql.functions import min, max, avg, variance, round
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator
from pyspark.ml.feature import StandardScaler
import pandas as pd
import matplotlib.pyplot as plt
import time

# ستون‌های مورد استفاده برای خوشه‌بندی
columns = ["Annual Income (k$)", "Spending Score (1-100)"]
data = data.select(columns)

# تبدیل داده‌ها به فرمت قابل پردازش
assembler = VectorAssembler(inputCols=columns, outputCol="features")
final_data = assembler.transform(data)

# استانداردسازی داده‌ها
scaler = StandardScaler(inputCol="features", outputCol="scaledFeatures")
scalerModel = scaler.fit(final_data)
final_data = scalerModel.transform(final_data)
final_data.select('scaledFeatures').show(5)

# K-Means خوشه‌بندی با روش
silhouette_score = []
evaluator = ClusteringEvaluator(predictionCol='prediction',
                                featuresCol='scaledFeatures',
                                metricName='silhouette',
                                distanceMeasure='squaredEuclidean')

kmeans = KMeans(featuresCol='scaledFeatures', k=3)
model = kmeans.fit(final_data)

# مراکز خوشه‌ها
centers = model.clusterCenters()

# پیش‌بینی خوشه‌ها برای داده‌ها
predictions = model.transform(final_data)

# امتیاز سیلوئت برای k=3
print(f"Silhouette Score for k=3 is {evaluator.evaluate(predictions)}")

# نمایش مراکز خوشه‌ها
print("Cluster Centers: ")
for center in centers:
    print(center)

# نمایش بخشی از داده‌ها با برجسته خوشه
predictions.select(*columns).show(5)

```

نتایج: به ترتیب کد نتایج زیر در خروجی نمایش داده شد:

```
final_data.select('scaledFeatures').show(5)
```

```
24/12/27 15:29:01 INFO DAGScheduler: Job 6 finished: showString at NativeMethodAccessorImpl.java:0, took 7.300930 s
24/12/27 15:29:01 INFO CodeGenerator: Code generated in 43.659877 ms
```

```
+-----+
| scaledFeatures |
+-----+
|[0.57110829030364...|
|[0.57110829030364...|
|[0.60918217632388...|
|[0.60918217632388...|
|[0.64725606234413...|
+-----+
```

only showing top 5 rows

```
print(f"Silhouette Score for k=3 is {evaluator.evaluate(predictions)}")
```

```
print("Cluster Centers: ")
```

```
for center in centers:
```

```
    print(center)
```

```
24/12/27 15:29:34 INFO DAGScheduler: Job 38 finished: collect at ClusteringMetrics.scala:102, took 0.320202 s
24/12/27 15:29:34 INFO CodeGenerator: Code generated in 30.133536 ms
24/12/27 15:29:34 INFO TorrentBroadcast: Destroying Broadcast(106) (from destroy at ClusteringMetrics.scala:413)
Silhouette Score for k=3 is 0.6288672765684974
Cluster Centers: 24/12/27 15:29:34 INFO BlockManagerInfo: Removed broadcast_106_piece0 on raspberrypi-dml0:46733 in memory (size: 578.0 B, free: 365.9 MiB)

[3.29485552 3.18036425]
[1.68113232 1.92960778]
[3.31242808 0.72149644]
```

```
predictions.select(*columns).show(5)
```

```
24/12/27 15:29:35 INFO DAGScheduler: Job 39 finished: showString at NativeMethodAccessorImpl.java:0, took 0.412473 s
24/12/27 15:29:35 INFO CodeGenerator: Code generated in 26.522283 ms
```

```
+-----+-----+
| Annual Income (k$) | Spending Score (1-100) |
+-----+-----+
| 15 | 39 |
| 15 | 81 |
| 16 | 6 |
| 16 | 77 |
| 17 | 40 |
+-----+-----+
```

د) در این بخش نمایش نتایج خوشه‌بندی مشتریان بر اساس درآمد سالانه و امتیاز مخارج به صورت نمودار با استفاده از الگوریتم K-Means هر خوشه با یک رنگ متمایز مشخص شده است.

```
# تبدیل DataFrame داده‌های پیش‌بینی شده به
predictions_pd = predictions.toPandas()

# رسم نمودار
plt.figure(figsize=(7, 7))
c = predictions_pd['prediction']
for cluster_id in c.unique():
    clusterData = predictions_pd[c == cluster_id]
    plt.scatter(clusterData["Annual Income (k$)"],
                clusterData["Spending Score (1-100)"],
                label=f'Cluster {cluster_id}', s=50)

plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("K-Means")
plt.legend()
plt.grid()
plt.show()

# ذخیره نمودار به عنوان فایل تصویر
plt.savefig("kmeans.jpg", dpi=300)
```

توضیح کد:

۱. تبدیل به **DataFrame**: داده‌های پیش‌بینی‌شده (شامل برچسب‌های خوشه‌بندی) به DataFrame تبدیل شدند.
۲. رسم نمودار: نقاط داده‌ها بر اساس درآمد سالانه و امتیاز مخارج در یک نمودار ترسیم شدند. هر خوشه با رنگ و برچسب متمایزی نمایش داده شده است.
۳. ذخیره تصویر: نمودار رسم‌شده در قالب فایل تصویری با نام `kmeans.jpg` ذخیره شد.

نتیجه:

نمودار زیر نمایش‌دهنده خوشه‌بندی مشتریان بر اساس الگوریتم K-Means است:

